```
    <div>
      {messages.length === 0 && (.
        <h1>
          <div>Hello, I'm ✳ Astra</div>
          <br />
          <span>Ask me anything you want</span>
        </h1>
      )}

      {messages.length > 0 &&
<ChatList messages={messages} isLoading={isLoading} />}

      <form ref={formRef} onSubmit={handleSubmit}>
        <Textarea
          ref={inputRef}
          placeholder="Type your message here..."
          value={inputValue}
          onChange={onInputChange}
          onKeyDown={onKeyDown}
        />
      </form>
      <AutoScroll trackVisibility />
    </div>
  );
};

export default ChatPage;
```

Conditional UI: shows a welcome message when empty

Conditional UI: renders the ChatList

Chat form: renders the text area, wired to state and handlers

Autoscroll: keeps the chat window pinned to the latest message

2.5.3 ChatPage

`<ChatPage/>` imports certain components for displaying past conversations located inside the components folder (`<ChatList/>`) and user input (`<Textarea/>`) and includes automatic scrolling support (via `<AutoScroll/>`). Additionally, it uses custom hooks (located in the hooks folder), which are reusable functions that encapsulate state and logic for specific tasks.

2.5.4 ChatList

The `<ChatList/>` component is responsible for a core UI concept: rendering dynamic lists. It takes an array of message objects as a prop and maps over them to display each conversation bubble. This pattern, in which a parent component manages a list of child components, is fundamental to building scalable UIs.

Nested within the `<ChatList/>` component is the `<ChatMessage/>` component. This smaller component renders each message in the conversation history. It takes a single message object as a prop and formats it appropriately, often distinguishing between user messages and AI messages (e.g., by applying different styling or showing an avatar) and a role prop that indicates whether the message came from the "user" or "assistant," and text contains the actual message content.